

Project Proposal

Course Code: CSE-416

Course Title: Software Project Design and Development

Submitted to: **Dr. Mahfida Amjad Dipa**

Group Members:

· **Zannatul Nayem** (0432320005101063)

· **Reon Ahmed** (0432320005101066)

SEC: 7B

Batch: 54

Semester: Autumn 2026

Project name: Automated MCQ Exam Portal.

1. Objectives

The objectives of the Automated MCQ Exam Portal are to establish a secure, maintainable, and academically useful web-based examination platform for students, teachers, and administrators.

1. **Secure portal:** Develop a secure role-based web portal that protects examination data and provides appropriate access for Student, Teacher, and Admin roles.
2. **Teacher-led examination management:** Enable teachers to create, manage, schedule, and conduct timed multiple-choice examinations through a structured web interface.
3. **Student assessment:** Enable students to attempt assigned examinations and receive automatically evaluated results after MCQ submission.
4. **Reusable question bank:** Maintain a reusable question bank with organized questions and categorization to support efficient examination preparation.
5. **Performance analytics:** Provide basic analytics that support performance review and help teachers identify examination and learner trends.
6. **Administrative control:** Support administration of users, courses, examinations, and permissions through dedicated management functions.
7. **Traceability:** Maintain audit logs for important system activities so that examination and administrative actions remain traceable.

2. Scope

In Scope

- Student, Teacher, and Admin roles with role-specific system capabilities.
- Authentication and role-based access control for protected portal functions.
- Course and user management for academic administration.
- Question-bank CRUD operations, organization, and categorization.
- Exam creation, configuration, scheduling, and management.
- Timed examination attempts, MCQ submission, automatic evaluation, and result publication.
- Basic analytics for examination and learner performance review.
- Audit logging of relevant user, examination, and administrative activities.
- Responsive web interface suitable for access through modern browsers.
- Technology stack: React + Vite frontend, NestJS backend, Prisma ORM, and Supabase PostgreSQL database.

Out of Scope

- Subjective or descriptive answer marking.
- Live video proctoring and biometric verification.
- Native mobile applications for Android or iOS.
- Advanced AI-generated question creation.

- Payment processing or fee collection.
- Institution-wide production deployment and operational support at institutional scale.

Scope boundary: The scope is limited to a functional student-project prototype suitable for demonstration and academic evaluation.

3. Feasibility

Technical Feasibility

React + Vite, NestJS, Prisma, and Supabase PostgreSQL are suitable, interoperable, and manageable technologies for a student team. The proposed capabilities—including timed examinations, role-based access, automatic evaluation, analytics, and audit logs—can be implemented using standard web technologies and established development practices. The modular separation between frontend, backend, ORM, and database also supports maintainable implementation and testing.

Economic Feasibility

Open-source frameworks and free or low-cost hosted services minimize expenditure for the project. The team can develop and demonstrate the prototype with limited infrastructure costs, while the estimated academic delivery budget is provided in Section 6.

Operational Feasibility

The portal is operationally feasible because students, teachers, and administrators can use role-specific functions through a responsive

browser-based interface. A modular architecture will support maintainability, troubleshooting, and future enhancement without requiring a complete redesign of the system.

Schedule Feasibility

The proposed work can be completed within an 8-week academic schedule through phased development, testing, documentation, and demonstration. The schedule allows parallel progress on frontend and backend foundations while reserving later phases for integration, quality assurance, deployment preparation, and final presentation.

4. WBS

WBS Code	Work Package	Key Activities/Deliverable
1.0	**Project Initiation and Planning**	Project baseline, planning package, and initial coordination.
1.1	Requirements and stakeholder analysis	Identify stakeholders, user needs, role expectations, and functional requirements.
1.2	Feasibility and proposal	Prepare the feasibility assessment and Lab-2 project proposal.

1.3	Initial plan and risk identification	Define the initial schedule, dependencies, assumptions, and project risks.
2.0	**System Analysis and Design**	Approved analysis and design baseline for implementation.
2.1	Use cases and role definitions	Document use cases and define Student, Teacher, and Admin responsibilities.
2.2	Database schema and Prisma model design	Design entities, relationships, constraints, and Prisma data models.
2.3	System architecture and API design	Define modular architecture, service boundaries, API endpoints, and data flows.
2.4	UI wireframes and navigation flow	Prepare responsive interface wireframes and role-specific navigation flows.
3.0	**Frontend Development**	React + Vite web interface and user-facing functionality.

3.1	React + Vite setup	Configure the frontend project, routing, shared components, and development structure.
3.2	Authentication and role-based UI	Implement sign-in flows, protected routes, and role-specific interface access.
3.3	Student exam interface and timer	Build examination navigation, timed attempt display, answer selection, and submission interface.
3.4	Teacher question-bank and exam-management interfaces	Develop question-bank CRUD, categorization, exam creation, configuration, and scheduling views.
3.5	Admin user/course/exam-management interfaces	Develop administrative screens for users, courses, examinations, and permissions.
3.6	Results and analytics views	Present published results and basic performance analytics for authorized users.
4.0	**Backend Development**	NestJS services, APIs, authorization, and examination logic.

4.1	NestJS setup and modular structure	Configure backend modules, shared services, validation, and project conventions.
4.2	Authentication and authorization	Implement authentication, authorization guards, and role-based permissions.
4.3	Question-bank, course, and exam APIs	Provide APIs for question, course, examination, configuration, and management operations.
4.4	Exam-attempt and timer-handling services	Manage attempts, timing rules, answer persistence, submission, and attempt state.
4.5	Automatic evaluation and results APIs	Evaluate MCQ submissions and expose result publication and retrieval services.
4.6	Analytics and audit-log services	Implement basic analytics aggregation and traceable activity logging.
5.0	**Database and Integration**	Configured database, connected application layers, and validated system integration.

5.1	Supabase PostgreSQL provisioning	Provision the database environment, access settings, and project connection.
5.2	Prisma migrations and seed data	Apply migrations and prepare representative seed data for development and demonstration.
5.3	Frontend-backend integration	Connect user interfaces with backend APIs and verify end-to-end data flow.
5.4	Validation, error handling, and security checks	Apply input validation, consistent error handling, and baseline security checks.
6.0	**Testing and Quality Assurance**	Test evidence and quality improvements for the functional prototype.
6.1	Unit testing	Test individual services, utilities, components, and core examination rules.
6.2	Integration testing	Verify interactions among frontend, backend, ORM, database, and major APIs.

6.3	Role-based and workflow testing	Test Student, Teacher, and Admin permissions and representative workflows.
6.4	Usability, responsiveness, and performance checks	Review browser usability, responsive layouts, and baseline performance behavior.
6.5	Defect fixing and regression testing	Resolve identified defects and repeat relevant tests after changes.
7.0	**Deployment, Documentation, and Demonstration**	Demonstrable prototype, supporting documentation, and submission package.
7.1	Deployment/configuration	Prepare deployment configuration and a stable demonstration environment.
7.2	User and technical documentation	Prepare user guidance, technical notes, configuration details, and test evidence.
7.3	Final presentation and demonstration	Prepare the final presentation, walkthrough, and functional system demonstration.

7.4	Submission packaging	Organize source materials, documentation, and required academic submission files.

5. Gantt Chart

Activity	W1	W2	W3	W4	W5	W6	W7	W8
Project initiation and requirements	●	–	–	–	–	–	–	–
Feasibility, proposal, and risk planning	●	●	–	–	–	–	–	–
System analysis and UI/database/API design	–	●	●	–	–	–	–	–
Frontend foundation and authentication	–	–	●	●	–	–	–	–
Backend foundation and authentication	–	–	●	●	–	–	–	–

Question bank, course, and exam management	–	–	–	●	●	–	–	–
Timed attempts, automatic evaluation, results, and analytics	–	–	–	–	●	●	–	–
Admin functions and audit logs	–	–	–	–	–	●	–	–
Integration and system testing	–	–	–	–	–	●	●	–
Bug fixing, usability, and performance checks	–	–	–	–	–	–	●	–
Deployment, documentation, and demonstration preparation	–	–	–	–	–	–	–	●
Final submission and presentation	–	–	–	–	–	–	–	●

Note: The timeline is indicative, and activities may overlap where integration dependencies permit.

6. Budget

Item	Basis/Description	Estimated Cost (BDT)
Domain name and basic deployment allowance	1 year	BDT 1,500
Cloud/database/service contingency	Free-tier usage with contingency	BDT 1,000
Development and testing internet/data	Project period	BDT 3,000
Documentation and printing	Proposal and final submission	BDT 1,500
Presentation/demo materials	Slides, handouts, and backup media	BDT 1,000
Contingency reserve	Approximately 10% miscellaneous reserve	BDT 800
Total		**BDT 8,800**

Software frameworks and Supabase PostgreSQL are assumed to use open-source or free-tier resources, so the estimate mainly covers academic delivery and project operations.